

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a deep blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene, digital aesthetic.

DL for ENG

LECTURE 5.2

RNN, LSTM, Attention and Transformers

One question, two answers thirty years apart: how does information get from the start of a long sequence to the end of it?

Remus Teodorescu

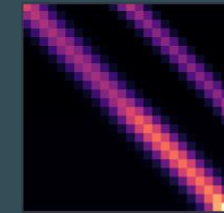
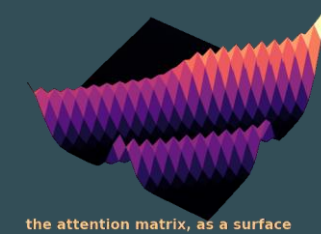
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

the RNN · the LSTM · attention · the transformer

FIVE DAYS OF SUBSTATION DEMAND



the diagonal ridge is recency

the second ridge is the same hour yesterday

seen from above

The series of Ex_05 notebook 04: the next hour, from the last twenty-four.

TWO ROUTES FROM HOUR 1 TO HOUR 24



recurrence — twenty-three steps, through twenty-three activations

Learning Objectives

SECTIONS, IN ORDER

- 1 **Sequences** — what makes a problem sequential, and why the last lecture's layers do not fit
- 2 **RNN** — what a recurrent network is; the hidden state, and the gradient over time
- 3 **LSTM** — what an LSTM is; the cell and its equations, the gates, sizing
- 4 **Attention** — what attention is; self-attention, the matrix, heads and position, what it adds
- 5 **Transformers** — the standard architecture, the layer, and the three model families
- 6 **Model selection** — choosing among notebook 04's four models: seed spread, residual check
- 7 **Challenges** — what goes wrong first, and what to check

WHAT WE SKIP, AND WHERE IT IS TAUGHT

training	gradients and backpropagation — L6.1
post-training	instruction tuning, RL — L6.2
the JetRacer	a sequence of frames — L11.1

UDL covers recurrence only in its ch. 12 notes: sections 2 and 3 cite the Wikipedia articles on the recurrent network and the LSTM; sections 4 and 5 follow UDL ch. 12.

Recap

THE TEN QUESTIONS OF L5.1 — CONVOLUTIONAL AND GRAPH NETWORKS

- 1** What kind of network is a convolutional network, and what function does it implement? *UDL ch. 10.1 – 10.3*
- 2** Describe the architecture of a convolutional network, layer by layer: what happens to resolution and channels? *UDL ch. 10.3 – 10.4; Ex_05 notebook 01*
- 3** How does a convolutional network differ from a dense network in connections, weights, parameters and symmetry? *UDL ch. 10.2; Ex_05 notebook 01*
- 4** Given those differences, what are convolutional networks used for, and which task needs equivariance, not invariance? *UDL ch. 10.1, 10.5*
- 5** Why does a residual connection help the gradient, and what does a block with nothing to add learn? *UDL ch. 11.1 – 11.2*
- 6** What kind of network is a graph network, and what function does it implement on node features and adjacency? *UDL ch. 13.1 – 13.3*
- 7** Describe the architecture of a graph network: the three steps of a layer, the readout, and K against the diameter? *UDL ch. 13.4, 13.7; Ex_05 notebook 02*
- 8** How does a graph network differ from a dense network, and what does permutation equivariance guarantee? *UDL ch. 13.2; Ex_05 notebook 03*
- 9** Given those differences, what are graph networks used for, and what is the difference between inductive and transductive? *UDL ch. 13.3, 13.6*
- 10** When is a graph network the wrong choice, and what did notebook 03 measure that shows it? *Ex_05 notebook 03*

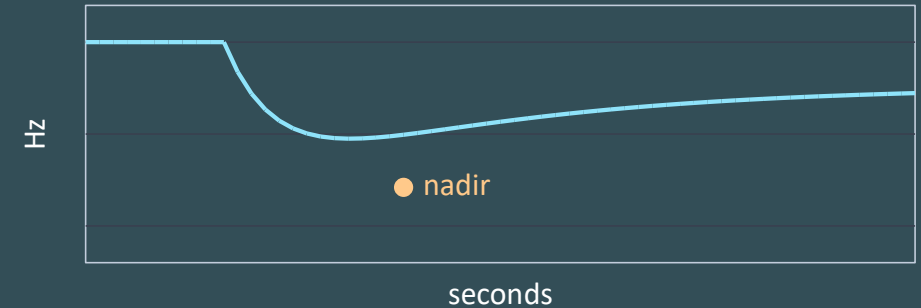
Sequences

ORDER MATTERS, AND THE LENGTH VARIES

- **examples** — a frequency trace, a load profile, a sensor stream
- **order matters** — shuffle the hours and the problem is destroyed
- **the length varies** — laps, runs and outages differ in length
- **convolution needs a fixed neighbourhood** — a sequence has none
- **a dense layer needs a fixed length** — 24 inputs, 24 weights

TALK · 2 MIN Look at the frequency trace on the right. Which earlier seconds does the nadir depend on, and why would padding every event to one length not make it a fixed-size input?

A GRID FREQUENCY EVENT



THE REQUIREMENT

A rule that reads one step at a time and keeps a summary of everything before it. That is a recurrence. The opposite of the six-bus network, where renumbering had to change nothing.

Recurrent Network

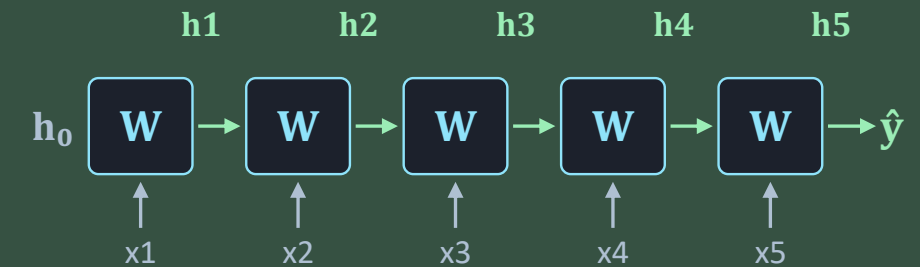
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a network with a state, fed back into itself at every step
- **function** — a sequence in, an output per step or at the end
- **architecture** — one cell, applied at every step; unrolled, T copies
- **applications** — forecasting, speech, control traces

AGAINST A DENSE NETWORK

	dense network	recurrent network
input	one vector, fixed length	a sequence of any length
memory	none between inputs	the hidden state h
weights	one set per input position	one cell, shared over time
parameters	grow with the window	independent of the length
order	an ordinary input index	built in: step t follows $t - 1$

THE ARCHITECTURE, UNROLLED



the same W in every copy — no new parameters as T grows

One cell reads x at step t together with the state from step $t - 1$ and writes the new state. The output can be read at every step or once at the end. Weight sharing a third time: position, node, and now time step.

Hidden State

A STATE, A RULE FOR UPDATING IT, AND UNROLLING

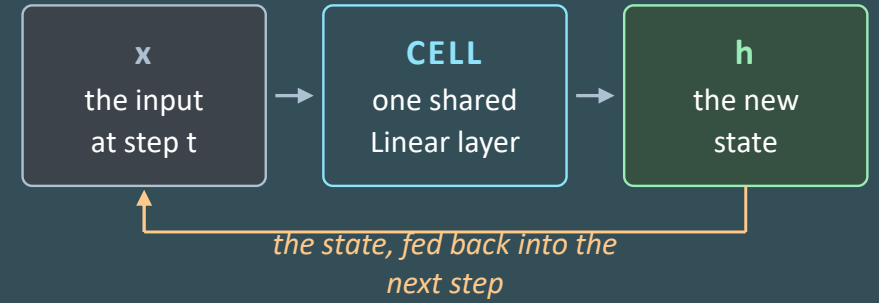
- **the hidden state** — a lossy summary of everything seen so far
- **each step** — combine input and state, apply tanh, update
- **one matrix on the state, one on the input** — the same every step
- **to train, unroll** — one copy of the cell per step, no new parameters
- **weight sharing, a third time** — position, node, and now time step

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

$$\hat{y} = \mathbf{W}_y \mathbf{h}_T + b_y$$

$\mathbf{x}_t$ the input at step t , $\mathbf{h}_t$ the state, $\mathbf{W}_h, \mathbf{W}_x$ shared every step, $\hat{y}$ read from the last state

THE LOOP



THE WHOLE NETWORK, AS A LOOP

$h = h_0$; for t in $1 \dots T$: $h = \tanh(W_x \cdot x[t] + W_h \cdot h + b)$. Then $\hat{y} = W_y \cdot h + b_y$. Training runs the loop, then backpropagates through every iteration of it.

Vanishing Gradient

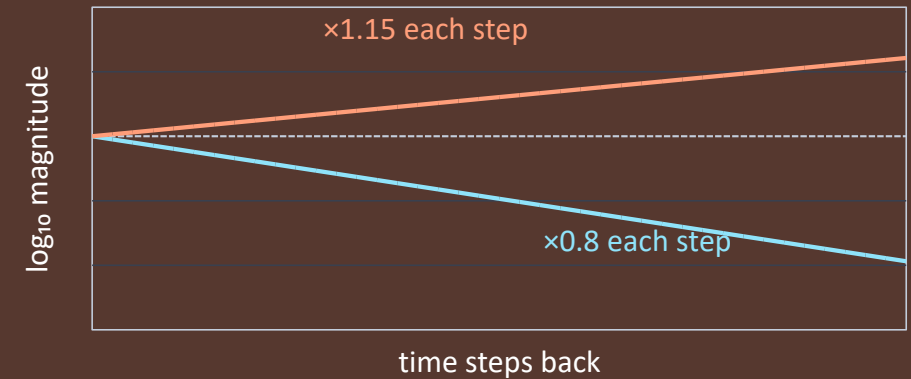
THE SAME MECHANISM AS DEPTH, ON THE TIME AXIS

- **the gradient through time** — a long product, one factor per step
- **twenty-four steps** — a twenty-four-layer network
- **worse than depth** — one matrix repeated, not different ones
- **below one it vanishes by ten steps** — above one it explodes
- **exploding has a fix** — clipping; vanishing has none

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{t=2}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

$\mathcal{L}$ the loss, $\mathbf{h}_1$ the first state, T the sequence length, the product is what vanishes

GRADIENT MAGNITUDE, BACK IN TIME



GRADIENT CLIPPING

If the gradient norm exceeds a threshold, rescale it. Three lines, and it cures the exploding case. Vanishing loses the information itself.

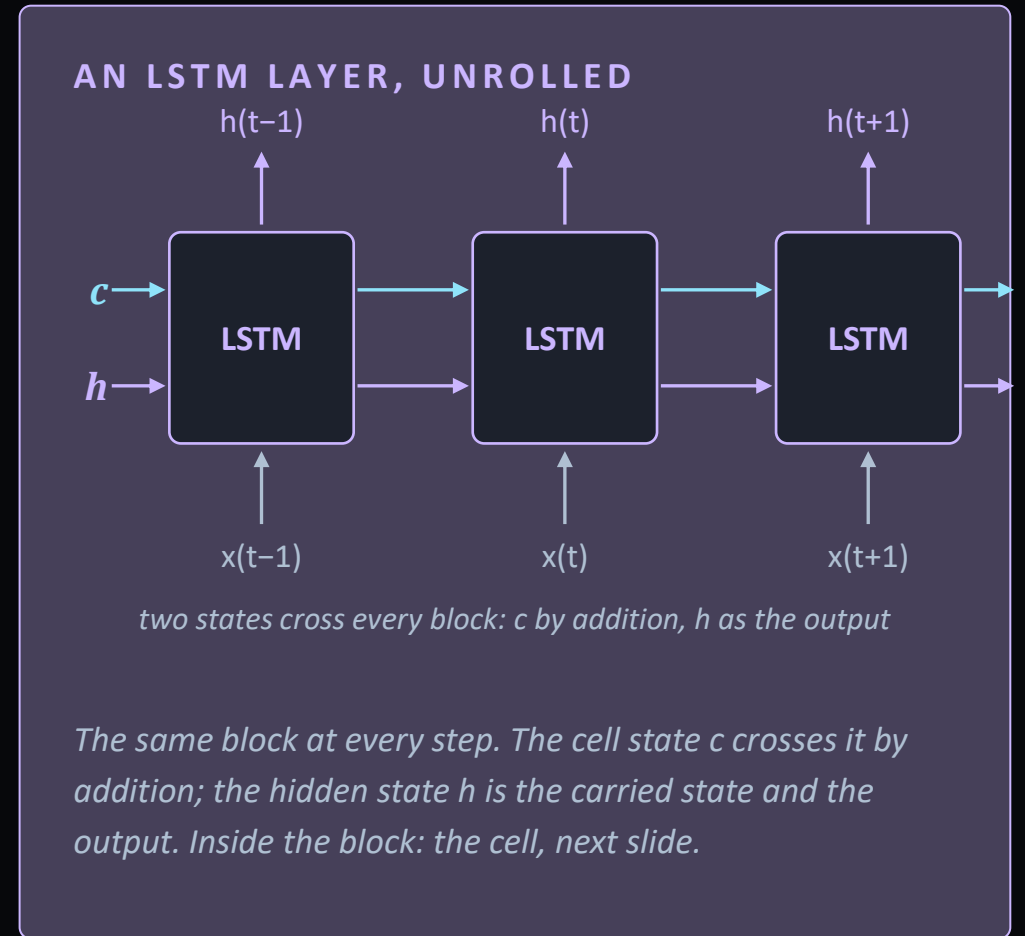
The LSTM

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a recurrent network with a gated cell state beside h
- **function** — a sequence in, an output per step or at the end
- **architecture** — a belt, three gates and a candidate; drawn right
- **applications** — forecasting, speech, text, state estimation

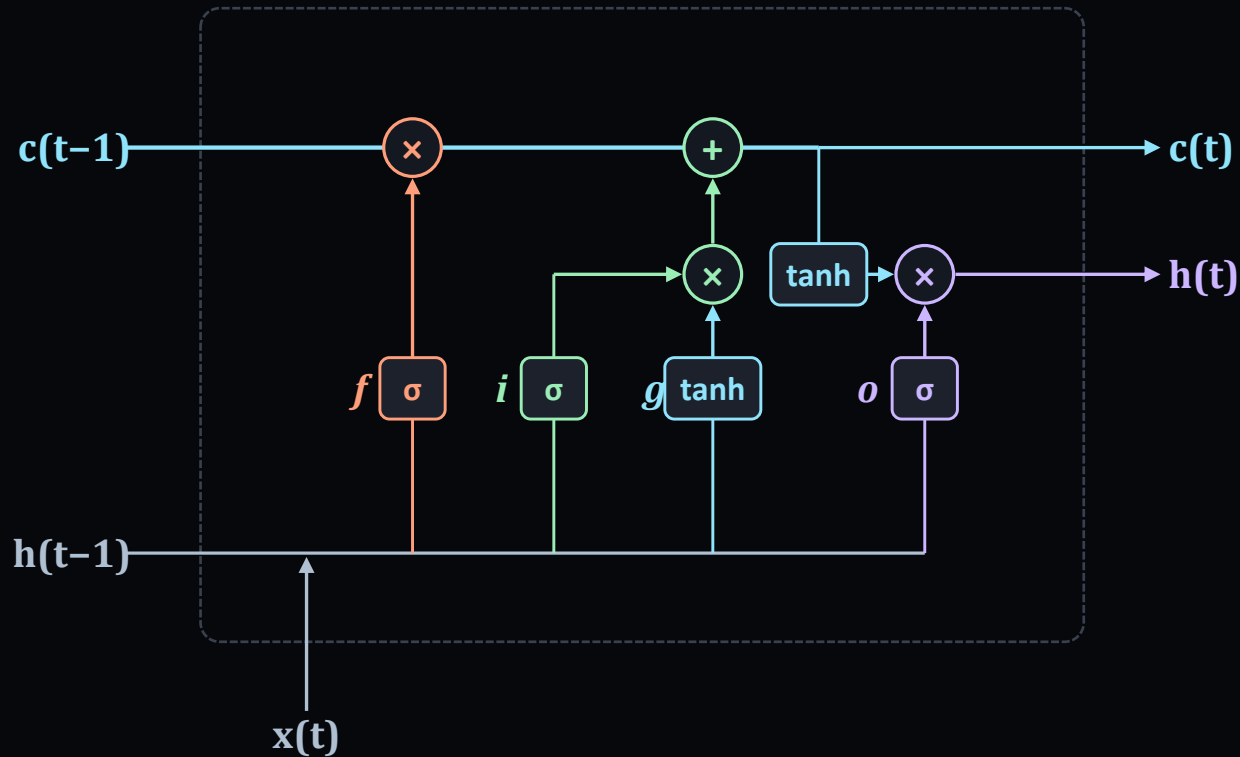
AGAINST THE PLAIN RECURRENCE

	plain recurrence	LSTM
state	one vector h , overwritten	h , and a cell state c beside it
update	tanh of a matrix product	addition on c , then gates
gradient in time	a matrix power: it vanishes	the forget gate, elementwise
reach	about ten steps	hundreds of steps
parameters	$H(C + H + 1)$	$4H(C + H + 1)$, four times as many



LSTM Architecture

THE CELL



The first four equations are one Linear layer of width $4H$ on $[x, h]$: $4H(C + H + 1)$ parameters for input width C and hidden width H .

THE SIX EQUATIONS, GATE BY GATE

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$$

forget gate — how much of the belt to keep

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$$

input gate — how much of the candidate to add

$$g_t = \tanh(W_g[x_t, h_{t-1}] + b_g)$$

candidate — what could be added

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$$

output gate — how much of the cell to expose

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

cell update — keep, then add

$$h_t = o_t \odot \tanh(c_t)$$

hidden state — the exposed cell, and the output

Cell State

A SECOND STATE, UPDATED BY ADDITION

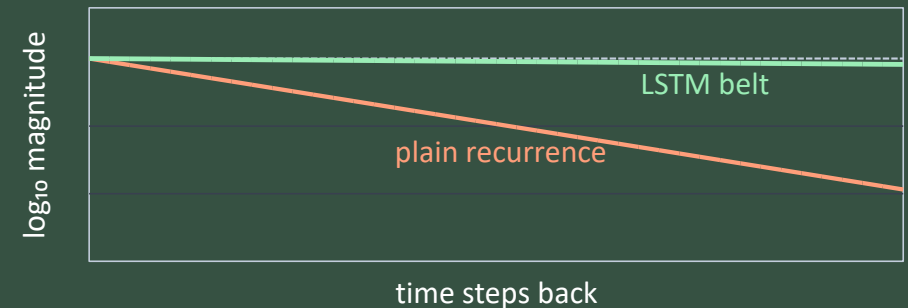
- a second state — the cell state c , alongside the hidden state h
- its update is addition — not a matrix multiplication
- left alone, c passes unchanged — its gradient is multiplied by one
- a residual connection in time — a path with no weights on it
- the rest of the LSTM — controls what goes on and comes off

THE CONVEYOR BELT



Left alone, c passes unchanged, and so does its gradient: a residual connection in time, 1997.

GRADIENT BACK IN TIME, GATE AT 0.99



Gates

A SIGMOID AND AN ELEMENTWISE MULTIPLICATION

- a **gate** — a vector in $(0, 1)$, from a sigmoid layer on $[x, h]$
- **forget gate f** — multiplies the belt: how much to keep
- **input gate i** — how much of a tanh candidate g to add
- **output gate o** — how much of the cell state to expose as h
- **memory and output separate** — c can hold a value and say it once

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f)$$

$\mathbf{f}_t$ the forget gate, σ squashes to $(0, 1)$, $[\mathbf{x}_t, \mathbf{h}_{t-1}]$ input and state stacked

FOUR LAYERS ON $[x, h]$

	layer	multiplies	decides
f	sigmoid	the belt c	how much to keep
i	sigmoid	the candidate g	how much to add
g	tanh	—	what could be added
o	sigmoid	$\tanh(c)$	how much to expose

Multiply, add, expose — the colours match the cell figure.

FORGET BIAS

Initialise the forget gate's bias to one: the unit remembers by default and learns when to forget.

Sizing an LSTM

WIDTH, DEPTH, DIRECTION, AND THE GRU

- **hidden width first** — sixteen to one hundred and twenty-eight
- **then depth** — two stacked layers is usually the most that helps
- **bidirectional only offline** — the backward pass reads the future
- **the GRU** — two gates instead of three, often as good and quicker

DECIDE · 2 MIN In the measured table on the right, the LSTM matches the perceptron's error at three times the parameters. For a substation controller, which of the three would you deploy, and why?

SENSIBLE DEFAULTS

hidden width	32 — grow only if underfitting
layers	1, or 2 at the most
clip the gradient	always, at norm 1
forget bias	initialise to 1

MEASURED IN EX_05 NOTEBOOK 04

model	parameters	held-out MSE
plain recurrent	305	0.00127
LSTM	1,169	0.00079
perceptron	417	0.00079

Attention

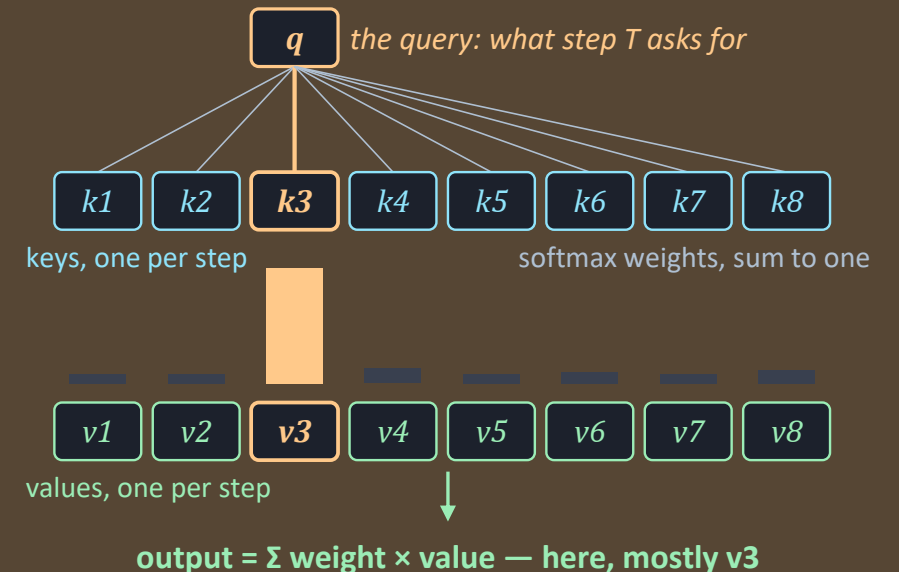
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a layer, not a recurrence; each position reads all others
- **function** — a weighted average of values; weights from query · key
- **architecture** — Q, K, V projections, softmax scores, weighted sum
- **applications** — language, long-window forecasting, vision, graphs

AGAINST A DENSE AND A RECURRENT NETWORK

	dense	recurrent	attention
input	fixed-length vector	sequence, in order	a set, any size
the past	not kept	carried in h	read directly
path, step 1 to T	—	T – 1 steps	one step
weights	fixed after training	fixed, shared in time	depend on the input
cost per layer	$C \cdot H$	$T \cdot H^2$	$T^2 \cdot H$

THE ARCHITECTURE: ONE LOOKUP



Scores from query and keys, softmax to weights, then the weighted values. Self-attention: every position is a query.

Self-Attention

EVERY POSITION READS EVERY OTHER, IN ONE STEP

- **do not carry it** — every position gets a direct path to every other
- **three vectors per position** — query, key, value, each learned
- **score query against key** — dot product, scale by $\sqrt{d}$, softmax
- **the output** — a weighted average of the values
- **all positions at once** — one parallel operation, no recurrence

$$\mathbf{Q} = \mathbf{XW}_q \quad \mathbf{K} = \mathbf{XW}_k \quad \mathbf{V} = \mathbf{XW}_v$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d}}\right)\mathbf{V}$$

$\mathbf{Q}, \mathbf{K}, \mathbf{V}$ queries, keys and values, d the width of one head, the weights sum to 1

WHAT RECURRENCE COSTS

path length	$T - 1$ steps from step 1 to step T
sequential	step t waits for step $t - 1$
bottleneck	one hidden vector holds the whole past

WHAT ATTENTION COSTS

path length	one, between any two positions
parallel	every position computed at once
the bill	quadratic in the sequence length

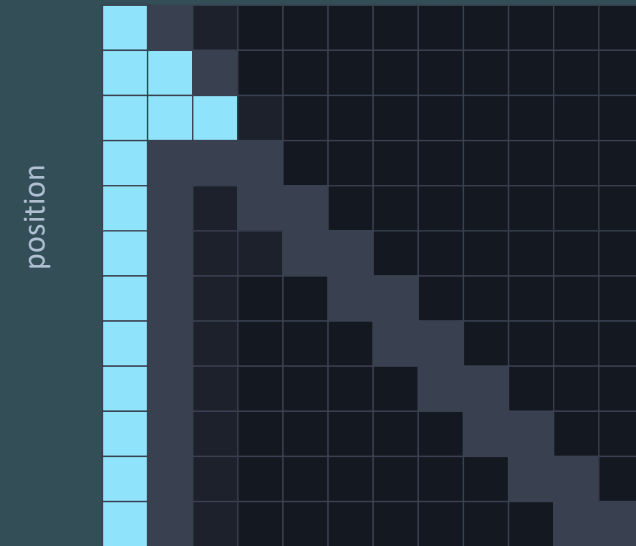
Attention Matrix

WHAT THE SOFTMAX PRODUCES

- a **square matrix** — one row per position, each row summing to one
- **readable** — which a dense layer is not
- **in notebook 04** — the same hour yesterday, and the last few hours
- **nobody told it to** — the daily cycle is the strongest signal
- a **weight says which values were used** — not why, not a cause

READ IT · 2 MIN In the matrix on the right, one row is brighter than the others at its left-hand end. Which hours is that position reading, and what in the data makes them worth reading?

MEAN ATTENTION, HELD-OUT WINDOWS



attends to position →

Bright is heavy. The bright left-hand columns are the same hour yesterday; the bright diagonal is the last few hours.

Heads and Position

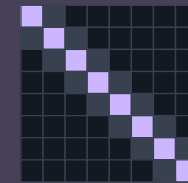
SEVERAL VIEWS, AND THE ORDER OF THE STEPS

- **one attention operation** — one notion of relevance
- **several heads** — several views in parallel, a slice of width each
- **attention is permutation equivariant** — order is invisible to it
- **on a graph that was the point** — on a time series it is a defect
- **positional encoding** — a vector per position, added to the input

$$\text{MultiHead}(\mathbf{X}) = [\mathbf{Z}_1, \dots, \mathbf{Z}_H] \mathbf{W}_o$$

head_{*i*} one attention, its own $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$, $[\cdot]$ concatenation, $\mathbf{W}_o$ mixes the heads

THREE HEADS, THREE VIEWS



recent hours



yesterday



a fixed lag

Different heads, different patterns of relevance, all computed at once.

THE SAME PROPERTY, OPPOSITE SIGN

Permutation equivariance made the graph network right for a power network. On a time series it is wrong, and the positional encoding is the patch.

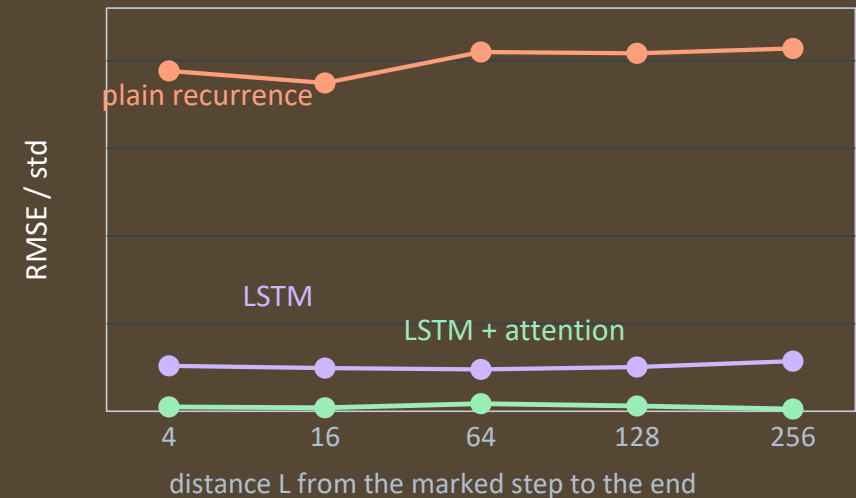
What Attention Adds

PATH LENGTH AND THE MEASURED ERROR

- **the value needed is L steps back** — recurrence carries it L steps
- **an attention layer reads that step directly** — one hop, any L
- **the weights depend on content** — it finds the step, not a fixed lag
- **measured** — recurrence 1.00, LSTM 0.13, with attention 0.01
- **notebook 04** — 24-hour window; a perceptron already matches

TALK · 2 MIN In the plot on the right the LSTM's error is flat in the distance and the attention layer's is near zero. What does the LSTM pay at every distance that attention does not?

HELD-OUT ERROR AGAINST THE DISTANCE



Report the input at a marked step, 4 to 256 steps before the end. Hidden width 32, 3,000 steps, seed 0; 1.0 is answering zero. The same LSTM, with one attention layer added, is 10 times more accurate.

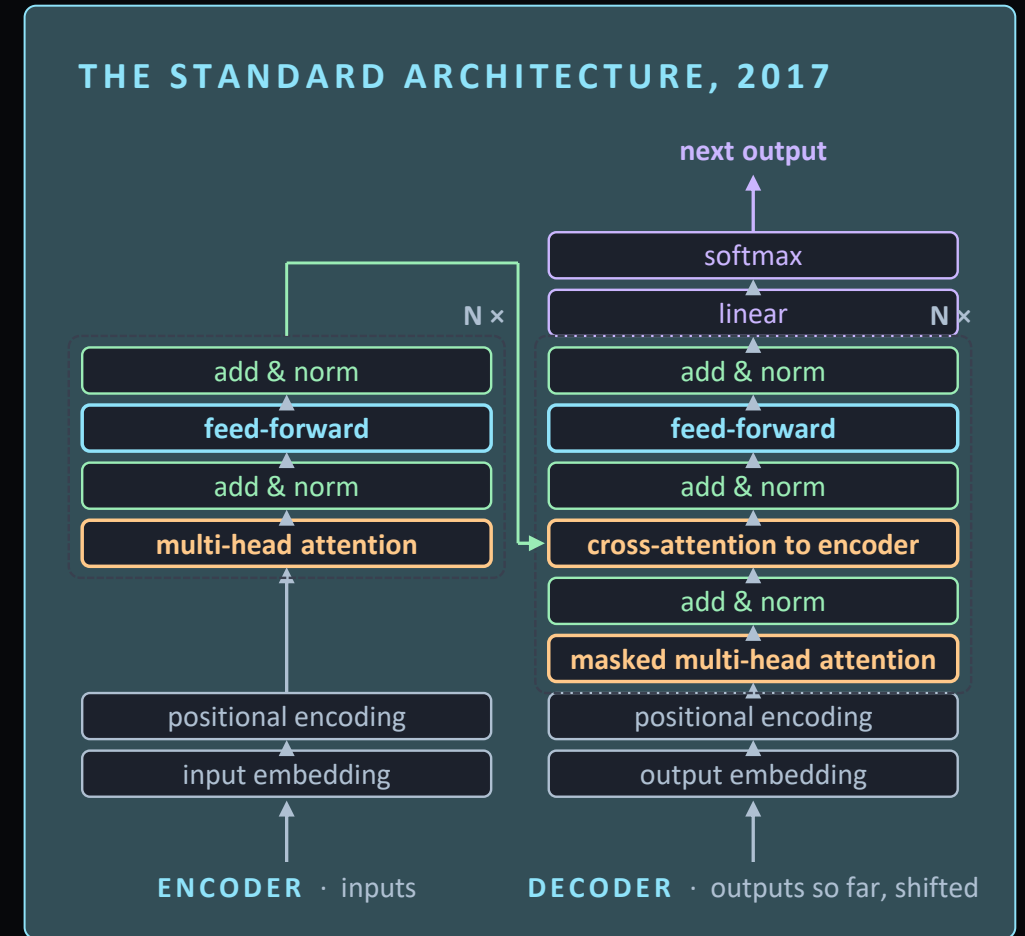
Transformer

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a stack of attention layers; no recurrence, no convolution
- **function** — a sequence in, a sequence out, all positions in parallel
- **architecture** — embedding and position, an encoder, a decoder
- **applications** — translation, language models, vision, time series

PROPERTIES, AGAINST THE RNN AND THE LSTM

	RNN, LSTM	transformer
path, step 1 to T	T – 1 steps	one step, any pair of positions
computation	sequential, step by step	parallel over the sequence
order	built into the recurrence	added: positional encoding
cost in T	linear	quadratic
depth in practice	one or two layers	six to a hundred layers



Transformer Layer

ATTENTION, A NETWORK PER POSITION, TWO RESIDUALS

- **multi-head attention** — the only place positions talk to each other
- **a feed-forward network** — applied to each position on its own
- residual connections and layer norm around both
- **stack it** — six layers in the original, ninety-six in a large one
- **nothing else changes with scale** — only the count

$$\mathbf{X} \leftarrow \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X})) \quad \mathbf{X} \leftarrow \mathbf{X} + \text{MLP}(\text{LN}(\mathbf{X}))$$

MHA multi-head attention, FFN a two-layer network per position, LN layer normalisation

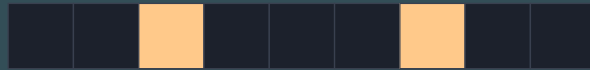
ONE LAYER, TOP TO BOTTOM



Everything but the attention treats each position on its own, which is why the whole layer parallelises.

Transformer Models

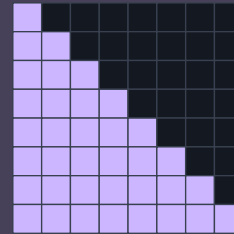
ENCODER — BERT



two positions hidden, predicted from the rest

Every position sees every other. Pre-trained by masked reconstruction, then a small head on your task.

DECODER — GPT



row i attends to columns $\leq i$

A position cannot see its own future. Every position is a training example.

VISION TRANSFORMER



one position per patch

Patches, flattened, are a sequence. Needs millions of images.

- **encoder** — reads the whole sequence, then answers about it
- **decoder** — predicts the next element — generation is that, repeated
- **the base model is not the assistant** — instruction tuning comes after

WHAT THIS SLIDE IS FOR

Enough to read a paper about one of these. Not enough to build one.

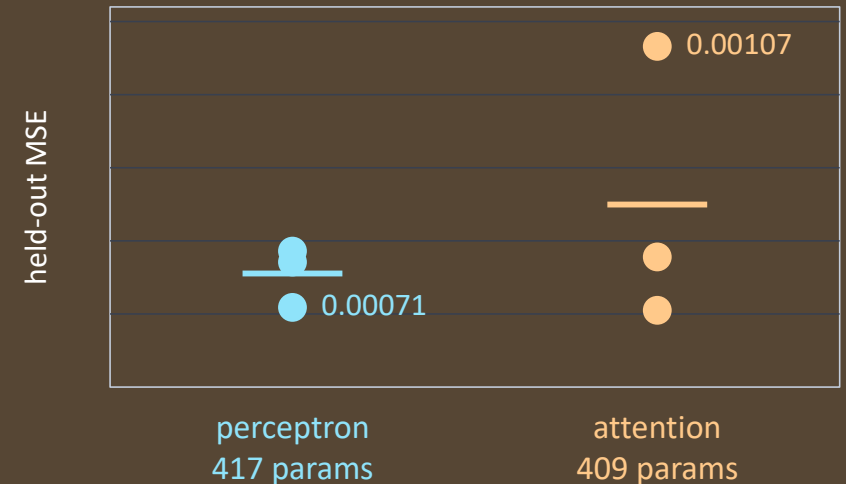
Seed Spread

FOUR MODELS FOR ONE FORECAST, AND THE FIRST CHECK

- **the choice** — perceptron, RNN, LSTM or attention, for notebook 04
- **seed zero** — attention nearly twenty per cent ahead
- **why seeds** — initialisation and shuffling are random draws
- **the check** — retrain from three seeds and plot the spread
- **the bands overlap** — the margin belonged to the seed
- **the rule** — a difference inside the seed spread is not a result

TALK · 2 MIN Look at the two columns of points on the right. From seed zero alone, which model would you have reported as better, and what do the other two seeds do to that claim?

THREE SEEDS EACH



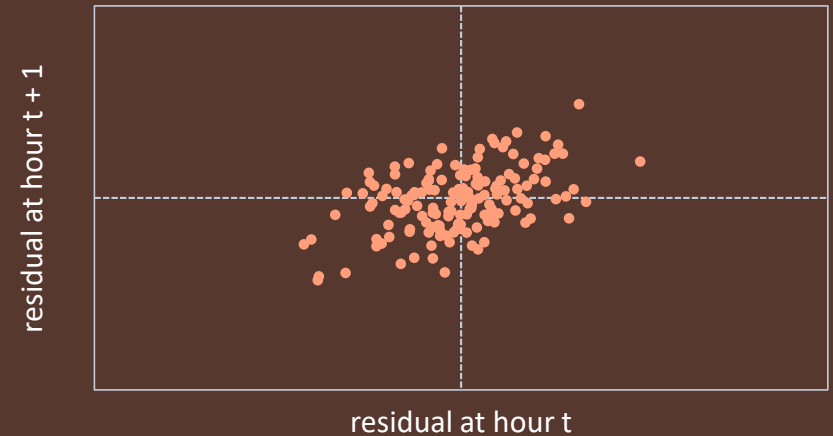
The distributions overlap. Seed zero of attention was the best run of the six; seed two was the worst.

Residual Check

THE SECOND CHECK, ON THE MODEL CHOSEN

- **why** — a low error can hide a failure the error does not show
- **the check** — plot each residual against the next one
- **the scatter leans upwards** — lag-one autocorrelation 0.5
- **predictable, and not predicted** — the noise is AR(1) at 0.75
- **nothing else showed this** — not the MSE, not the loss curves
- **the rule** — a model is chosen on its residuals, not on its MSE alone

RESIDUAL AT t AGAINST RESIDUAL AT $t + 1$



lag-1 autocorrelation: 0.50

If the model had extracted everything a twenty-four-hour window contains, this would be a formless cloud. It leans, and the lean is accuracy nobody collected.

Challenges

SHUFFLING BEFORE SPLITTING

overlapping windows share 23 of 24 values · split in time order · the held-out number is otherwise fiction

NO POSITIONAL ENCODING

attention cannot tell first from last · it still trains and reports a loss · add the encoding (Vaswani et al. 2017)

BIDIRECTIONAL ON A LIVE SIGNAL

the backward pass reads the future · offline analysis only · never on a controller

DATA AND ENERGY EFFICIENCY

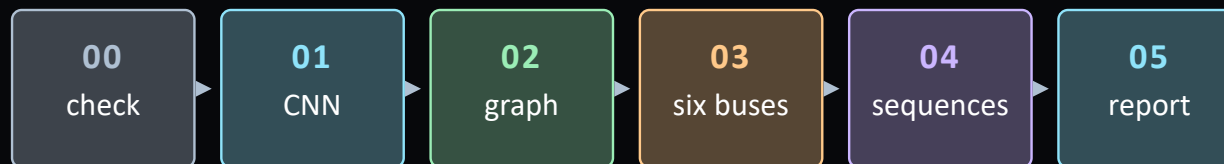
split	in time order
compare	against persistence first
plot	the residual autocorrelation
rerun	from three seeds
vary	the window length
test	on a different site or machine

The first four cost under ten minutes together and catch the leaked split, the meaningless baseline, the structured residual and the imaginary architecture win.

Exercise 5

SIX NOTEBOOKS, RUN IN ORDER; 04 IS THIS LECTURE'S

- **00** — environment check: versions and the four datasets
- **01** — convolution by hand, then a CNN on weld radiographs
- **02** — adjacency, neighbourhoods, message passing in NumPy
- **03** — node regression on the six buses; permutation measured
- **04** — RNN, LSTM and attention by hand; four models, one forecast
- **05** — the report: two marked questions, plus four from L3.2



THE QUESTIONS NOTEBOOK 04 ANSWERS

- § 5 does the LSTM repair the plain recurrence's error, and at what parameter cost?
- § 7 does attention's lead survive three seeds?
- § 9 what does the residual autocorrelation say that the MSE did not?

REPORT QUESTION

Four models within a factor of two of each other. Which would you deploy on a substation controller, and which single number decided it?

Key Takeaways

1 RNN — the recurrent network

A state fed back at every step and one cell shared over time, so the parameter count is free of the length. Its gradient through time is a power of one matrix, so it vanishes within about ten steps: the RNN cannot carry information far.

2 LSTM — the gated cell state

The same recurrence with a second state c , updated by addition and controlled by the forget, input and output gates. The derivative along c is the forget gate, elementwise, so the reach grows to hundreds of steps, at four times the parameters.

3 Attention — reading, not carrying

A layer, not a recurrence: each position scores a query against every key and averages the values with softmax weights that depend on the input. Path length one, parallel, quadratic cost. Measured: the LSTM carries a value at 0.13 error; one attention layer reads it at 0.01.

4 Transformer — attention stacked

Embedding plus positional encoding, then N layers of multi-head attention and a per-position feed-forward network, each with a residual and layer norm; an encoder reads, a decoder generates. BERT, GPT and the vision transformer are that stack trained three ways.

5 Model selection — two checks

Between notebook 04's perceptron, RNN, LSTM and attention: retrain from three seeds before believing a difference, and plot the winner's residuals. A margin inside the seed spread is not a result; a leaning residual scatter is accuracy left uncollected.

Next — L6.1: loss functions and gradients.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What kind of network is a recurrent network, and what function does it implement on a sequence? *Wikipedia, Recurrent neural network*
- 2 Describe its architecture: the hidden state, the unrolled network, and why the gradient over time vanishes? *Wikipedia, Recurrent neural network; Vanishing gradient problem*
- 3 What kind of network is an LSTM, and how does it differ from the plain recurrence in state, update and reach? *Wikipedia, Long short-term memory*
- 4 Describe the LSTM cell: the belt, the three gates, the candidate, and the six equations? *Wikipedia, Long short-term memory; Ex_05 notebook 04*
- 5 Given that, what is an LSTM used for, how is it sized, and why initialise the forget bias to one? *Wikipedia, Long short-term memory; Ex_05 notebook 04*
- 6 What kind of layer is attention, and what function do queries, keys and values implement? *UDL ch. 12.1 – 12.2*
- 7 Describe its architecture: the attention matrix, several heads, and why a positional encoding is needed? *UDL ch. 12.2 – 12.3; Ex_05 notebook 04*
- 8 How does attention differ from recurrence and from a dense layer, and what did adding a layer measure to? *UDL ch. 12.1; tools/deck/make_attention_fig.py*
- 9 Describe the standard transformer: embedding and position, encoder, decoder, one layer; what are BERT, GPT and ViT for? *UDL ch. 12.4 – 12.7*
- 10 A model wins by ten per cent on one run. What do you check first, and what does the residual autocorrelation add? *Ex_05 notebook 04, sections 7 and 9*

NEXT

L6.1 — Loss Functions and Gradients

Every architecture this course uses is now in hand: dense, convolutional, graph, recurrent and attention. Next: where the loss comes from, and how the parameters move.

BEFORE THE NEXT SESSION

read	UDL ch. 5.1 – 5.3
run	Ex_05 notebooks 04 and 05
bring	your three-seed spread
think about	which model to deploy, on what number

Bring the seed spread. Two measured cases now where the fashionable architecture did not win, and that is worth more than either architecture.